

SevaSetu — Product Requirements Document & Technical Implementation Specification

Version 2.0 (revised) · Tagline: Right Volunteer. Right Place. Right Impact. **Problem Statement:** PS 09 — Community Volunteer Coordination Platform **Audience:** Hackathon development team

Core flow: Need → Match → Schedule → Deploy → Track → Learn

SevaSetu is designed as a scalable platform, but the hackathon MVP intentionally implements a focused subset of the complete vision.

Label convention

Label	Meaning
[C]	Confirmed requirement
[P]	Proposed implementation (team may change)
[F]	Future scope
[TBD]	To be decided

What changed from v1: MVP re-tiered (demo-critical / secondary / future); Event established as the central entity; emergency flow rebuilt around real-time in-app alerts (email is a fallback); Agent boundaries tightened; Karma and Reliability separated; matching hierarchy simplified; "NGO rating" reframed as *NGO Transparency & Activity Indicators*; Golden User Journey, competitive insights, critical-decisions list and consistency audit added; document split into Part A (product) and Part B (technical). All strong v1 features, workflows and technical details are preserved.

Contents

PART A — PRODUCT REQUIREMENTS	A1 Product Overview · A2 Problem Statement · A3 Vision & Core Value Proposition · A4 Core Workflow · A5 Event as the Central Entity · A6 Target Users, Roles & Permissions · A7 User Journeys (incl. Golden User Journey) · A8 Feature Requirements (tiered) · A9 Functional Requirements & Acceptance Criteria · A10 User Stories · A11 MVP Scope · A12 Future Scope · A13 Success Metrics · A14 Competitive & Research Insights · A15 Risks · A16 Business Model · A17 Critical Decisions · A18 Hackathon Demonstration Flow
PART B — TECHNICAL IMPLEMENTATION SPECIFICATION	B1 System Architecture · B2 Technology Stack · B3 Database Architecture · B4 Data Model · B5 Matching Architecture · B6 Scheduling Architecture · B7 Emergency Architecture · B8 Agent Architecture · B9 API Design · B10 Authentication & Authorization · B11 Security & Privacy · B12 Scalability, Performance & Monitoring · B13 Deployment · B14 Technical Risks · B15 Development Roadmap
Appendices:	A Checklist · B Priority Matrix · C Database Responsibility · D Consistency Audit

PART A — PRODUCT REQUIREMENTS

A1. Product Overview

SevaSetu is an intelligent **Community Volunteer Coordination Platform**. It connects NGOs, affiliated volunteers, independent/freelance community volunteers, donors/public users and communities through:

- intelligent, explainable volunteer matching
- scheduling and roster generation
- deployment (including emergency mobilization)
- verified participation and attendance
- **Karma-based verified contribution history**
- **NGO Transparency & Activity Indicators**
- analytics and retention insights
- the **SevaSetu Event Intelligence Agent**

For NGOs: recruit suitable volunteers, match them to tasks, create and manage events, generate rosters, manage shifts, track attendance and task completion, mobilize quickly in emergencies, understand participation and retention, build credibility through transparency, and reach new independent volunteers.

For volunteers: discover NGO events and organizations aligned with their interests, participate in public events, build a verified record of community service, earn Karma, track hours and achievements, and build connections with social organizations.

A2. Problem Statement

PS 09 — Community Volunteer Coordination Platform [C]: create an intelligent volunteer operations platform that matches, dispatches, schedules and tracks skilled volunteers for NGO events, relief drives, community initiatives and emergency response.

#	Problem area	Where addressed
1	Multi-criteria skill & geo-matching (skills, certifications, availability, experience, proximity, reliability)	A9.4, B5
2	Reliability & reputation (attendance, completion, history) feeding future matching	A9.7, A9.8
3	Rapid emergency mobilization (broadcast, confirmation, specialized groups, backups, deployment)	A9.6, B7
4	Automated shift & roster generation	A9.5, B6
5	Retention & engagement analytics (hours, milestones, recognition, burnout indicators)	A9.12

A3. Product Vision & Core Value Proposition

Vision: a trusted bridge ("Seva" = service, "Setu" = bridge) between people who want to help and organizations that need help — where every contribution is verified, every match is explainable, and every NGO's activity is visible.

Core value proposition

For	Value
NGOs	Less manual coordination; the right volunteers for each task; fewer roster gaps; verified participation records; a public activity profile
Independent volunteers	A way in without joining an NGO; discovery of events and aligned NGOs; a verified service history
Affiliated volunteers	Fair, explainable assignments; recognition; clear view of hours and impact
Donors / public	Transparency and activity information to make informed decisions
Communities	Faster, better-staffed community initiatives and emergency response

Goals [C/P]

1. Reduce manual coordination effort for NGOs.
2. Improve match quality using more than skills alone.
3. Let unaffiliated people discover and take part in public events.
4. Give volunteers a verified service record (Karma, hours, achievements).
5. Enable rapid, skill- and location-aware emergency deployment with humans in control.
6. Give donors and volunteers transparent activity information.
7. Provide an AI assistant that recommends — never decides.
8. Ship a focused, working MVP before adding breadth.

Non-goals [C]: government schemes or benefits tied to Karma (intentionally removed); promises of employment or guaranteed NGO positions; declaring any NGO "good", "bad" or fraudulent; legal certification of NGOs; autonomous AI decisions; SMS/WhatsApp (TBD/future).

Product principles [C]

1. Agent recommends, human decides.
2. Do not overclaim AI capabilities; recommendations are explainable where possible.
3. Humans are responsible for final emergency decisions.
4. Karma is not a government credential.
5. Karma $\neq$ Reliability.
6. NGO indicators are platform-derived, not a certification or guarantee.
7. Protect volunteer privacy; never expose sensitive personal data publicly.
8. Supabase is the core source of truth; Firebase holds agent context and real-time state only.

9. Modular and scalable; prioritize a working MVP.

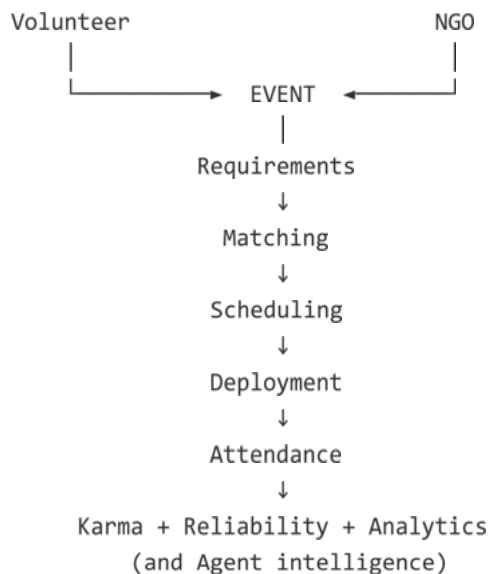
A4. Core Workflow: Need → Match → Schedule → Deploy → Track → Learn

Stage	What happens	Key modules	Primary data (Supabase)
Need	NGO defines an event or emergency with requirements	Event management, Emergency	events, event_requirements, shifts
Match	Eligible volunteers are filtered and ranked with explanations	Matching engine	applications, assignments (proposed)
Schedule	Roster generated; conflicts surfaced; backups allocated	Roster generator	rosters, assignments
Deploy	Volunteers confirm; NGO deploys confirmed volunteers (emergency: real-time alert → confirm → deploy)	Notifications, Emergency	assignments, emergency_responses
Track	Attendance/check-in and task completion verified	Attendance	attendance, tasks
Learn	Karma, reliability, analytics, and Agent insights updated from verified outcomes	Karma, Reliability, Analytics, Agent	karma_transactions, reliability_records, analytics

The flow is a loop: *Learn* outputs (reliability, history, analytics, Agent insights) improve the next *Match*.

A5. The Event: SevaSetu's Central Operational Entity

Every operational workflow in SevaSetu is anchored to an **Event** (an emergency is modelled as a special urgent event-type request that follows the same lifecycle, see B7).



The Event connects:

Connected concept	How the Event links it
NGO requirements	event_requirements and shifts belong to an event
Volunteer participation	applications (registration) reference the event
Matching	Matching always runs <i>for an event/shift</i>
Scheduling	Rosters are per event
Deployment	Assignments and confirmations are per event shift
Attendance	Attendance records reference an assignment within an event
Karma	Karma transactions reference the event and verified attendance
Reliability	Reliability records reference the assignment outcome in an event
Analytics	Metrics aggregate over events
Agent intelligence	Agent sessions are scoped to an event (or event-type history)

A6. Target Users, Roles & Permissions

Persona	Description	Primary needs
NGO Admin	Owns NGO profile, events, rosters, emergencies, transparency data, analytics	Recruit, staff, mobilize, show credibility
NGO Coordinator	Day-to-day event and volunteer coordination	Run events, review matches, attendance
Independent Volunteer / Freelancer	Not affiliated with an NGO; participates in public events temporarily	Find events, build verified history, discover aligned NGOs
Affiliated Volunteer	Ongoing relationship with one or more NGOs	Assignments, schedule, hours, recognition
Donor / Public User	Evaluating NGOs	Transparency and activity indicators
Event Intelligence Agent	Non-human assistant	Context in, recommendations out
Platform Administrator	Operates SevaSetu	Moderation, support, aggregate metrics [TBD scope]

Permissions matrix (authentication via Clerk; enforcement in Express, with Supabase RLS as defence in depth [PI])

Capability	Public/Donor	Independent Vol.	Affiliated Vol.	NGO Coord.	NGO Admin	Platform Admin
Browse NGO public profiles & indicators	✓	✓	✓	✓	✓	✓
Browse public events	✓	✓	✓	✓	✓	✓
Own volunteer profile	–	✓	✓	–	–	–
Register for events	–	✓ (public)	✓	–	–	–
Respond to assignments / emergency alerts	–	✓	✓	–	–	–
Create/edit events, requirements, shifts	–	–	–	✓ (own NGO)	✓	–
Run matching, generate/finalize roster	–	–	–	✓	✓	–
Mark attendance / task completion	–	–	–	✓	✓	–
Create emergency	–	–	–	[TBD, see A17]	✓	–
Deploy confirmed emergency volunteers	–	–	–	✓	✓	–
Edit NGO profile & transparency records	–	–	–	–	✓	–
Manage NGO members	–	–	–	–	✓	–
NGO analytics	–	–	–	✓	✓	✓ (aggregate)
Agent (volunteer mode)	–	✓	✓	–	–	–
Agent (NGO mode)	–	–	–	✓	✓	–
Moderate/suspend	–	–	–	–	–	✓

accounts

Boundaries [C/P]: NGO staff see only their NGO's data and only volunteers who applied, are assigned, are affiliated, or appear in privacy-limited matching results. Public users never see private volunteer data. Affiliation is a relationship record, not a separate account type, so one person can be independent for one NGO and affiliated with another.

A7. User Journeys

A7.1 Golden User Journey — Independent Volunteer

This is the primary product story and the basis of the hackathon demo.



Step	System behaviour	Event link / data
Creates profile	Clerk sign-up; profile saved to Supabase	users , volunteer_profiles
Adds skills, interests, availability, location	Controlled skill list; weekly availability; coarse location geocoded	volunteer_skills, availability
Discovers public	Filters by cause, date, distance	events (public)

events

Views NGO Transparency & Activity Profile	Indicators and NGO-provided info with disclaimer	ngo_transparency_records , ngo_indicators
Selects an event	Event details and requirements shown; Agent can explain preparation	events , event_requirements
Matching evaluates eligibility and fit	Eligibility gates, then ranked fit with "Why matched?" explanation; shown to NGO (and a fit indicator may be shown to volunteer [P])	matching output
Registers / gets assigned	Registration recorded; coordinator confirms	applications , assignments
Roster generated	Shift allocation, conflicts, backups; human finalizes	rosters
Attends event	Check-in	attendance
Attendance verified	NGO user verifies	attendance.verified_by
Karma recorded	Deterministic ledger entry	karma_transactions
Service history updated	Hours, event, tasks appear on profile	derived views
Discovers more NGOs/events	Recommendations aligned to causes and history	event/NGO discovery
Long-term connections	NGO may invite to affiliate; volunteer accepts	ngo_affiliations

Why it matters: the journey shows how SevaSetu converts an unaffiliated individual into an active participant in the community ecosystem — without requiring upfront NGO membership — while giving NGOs a pipeline of verified, vetted-by-participation volunteers.

Lifecycle [C]: Independent Volunteer → Public Event → Participation → Verified Contribution → Relationship Building → Potential NGO Affiliation.

A7.2 NGO Journey

Create NGO profile → create event → define requirements and shifts → set capacity → publish → review registrations and matches → generate and finalize roster → notify volunteers → track attendance → close event (Karma, reliability, analytics updated) → review Agent insights and dashboard.

A7.3 Emergency Journey (secondary scenario)

Authorized NGO user creates emergency → backend validates authorization → real-time in-app alert to eligible specialized volunteers → volunteers confirm → live confirmation count → backups considered →

NGO coordinator deploys confirmed volunteers → check-in → completion (details in B7).

A7.4 Karma and Reliability — the distinction

	Karma	Reliability
Measures	Verified community contribution	How consistently a volunteer fulfils assigned commitments
Examples	Verified attendance, completed assignments, service hours, participation history, achievements/milestones	Attendance rate, no-show rate, cancellation history, task completion, participation consistency
Direction	Accumulates (ledger)	Reflects recent and overall behaviour; can rise and fall
Shown as	Points + milestones	Labelled indicator with counts, or "Not enough history"
Role in matching	Not a direct matching input — never dominates matching	One factor in matching (lowest-priority in the hierarchy)
Purpose	Credibility and recognition	Planning confidence for coordinators

High Karma does not automatically mean high reliability. A volunteer may have participated in many events historically and so hold high Karma, while recent attendance problems may result in a lower reliability indicator.

A8. Feature Requirements (Tiered)

Tier A — Demo-Critical MVP (must work in the demonstration)

#	Feature	Notes
1	Authentication & role management	Clerk; roles per A6
2	Volunteer profile	Skills, certifications, interests, availability, location
3	NGO profile	Info, causes, members
4	Event creation	Requirements, shifts, capacity, visibility
5	Public event discovery	Cause/date/distance filters
6	Volunteer registration	Independent volunteers can register for public events
7	Multi-criteria volunteer matching	Eligibility → skills/certs → availability → proximity → experience → reliability, with explanations
8	Automated/basic roster generation	Conflicts surfaced; backups where practical

9	Attendance/check-in	NGO-verified
10	Karma calculation & verified contribution history	Deterministic ledger
11	NGO Transparency & Activity Profile	Indicators + disclaimers
12	Event Intelligence Agent (basic)	Event guidance, preparation checklist, risk and skill suggestions

Tier B — Secondary / Demo-Optional (if time permits)

- Advanced reliability analytics
- Advanced retention analytics
- Burnout indicators
- Emergency mobilization enhancements (live confirmation counts, backup proposals, specialized groups, check-in tracking)
- Advanced notification workflows (n8n-driven reminders, schedule-change notices)
- Advanced analytics dashboards

A *basic* emergency alert → confirm → deploy path and a *basic* dashboard are the optional secondary demo scenario; the enhancements above sit in this tier.

Tier C — Future Scope

See A12. Not removed from the product vision, only separated from the minimum working demonstration.

A9. Functional Requirements & Acceptance Criteria

Requirement IDs: FR-<area>-<n> . Tier tags: **(A)** demo-critical, **(B)** secondary, **(C)** future.

A9.1 Authentication & Identity (A)

- FR-AUTH-1: Users sign in via Clerk; the backend verifies the Clerk token on every protected request.
- FR-AUTH-2: Roles: volunteer, ngo_coordinator, ngo_admin, platform_admin; public browsing needs no login.
- FR-AUTH-3 [P]: A `users` row is created in Supabase keyed by Clerk user ID (webhook or first-request sync).
- FR-AUTH-4: NGO roles are tied to a specific NGO membership.

AC: Given a volunteer token, when calling an NGO-admin endpoint, then the API returns 403. Given a new Clerk user, when first authenticated, then a Supabase `users` row exists.

A9.2 Volunteer Management (A)

Onboarding: Registration → Profile → Skills → Certifications → Availability → Location → Preferences.

Profile field

Notes

Basic info	Name, optional photo, bio
Skills	Controlled list [P: seeded — e.g., first aid, logistics, teaching, medical, driving, languages]
Certifications	Name, issuer, expiry, optional document
Experience	Free text and past roles; history auto-populated from verified events
Availability	Recurring weekly windows plus exceptions
Location	Coarse home area for proximity; precise coordinates restricted (see B11)
Preferred causes / interests	Multi-select
Max travel distance	Volunteer-set
History & metrics	Participation, attendance, tasks, hours, Karma, achievements, reliability indicator

Certification verification approach: **Must decide (A17)**; MVP proposal: self-declared plus an "NGO-verified" flag any NGO can set after seeing the credential at an event. [P]

AC: Given a new user completes profile steps, then the profile is saved to Supabase and the volunteer is eligible for matching. Given availability is edited, then subsequent matching and rostering use it.

A9.3 NGO Management & Event Management (A)

- NGO profile: name, description, causes, region, contact, logo; members (admin/coordinator); affiliated volunteers; event participants (including independent volunteers); optional affiliation invites (volunteer must accept). [P]
- Event: title, description, cause, venue/geo, start/end, visibility (public / NGO-only), capacity, status. Lifecycle: Draft → Published → In Progress → Completed → Cancelled. [P]
- Requirements: role, headcount, required skills, required certifications, minimum experience [P]. Shifts: start/end, headcount, linked requirement.
- Public registration approval mode (auto up to capacity vs. NGO review): MVP proposal — NGO selects per event. Waitlist when full. [P]

AC: Given a coordinator completes fields, requirements and shifts and publishes, then the event is discoverable publicly with status Published. Given the event is at capacity, then further registrants are waitlisted or shown "full". Given a coordinator of NGO A requests NGO B data, then 403.

A9.4 Intelligent Volunteer Matching (A)

Workflow: Event requirements → Candidate filtering → Multi-criteria matching → Recommendations → NGO confirmation → Assignment.

Matching hierarchy (conceptual order) [C]

1. Eligibility

2. Required skills / certifications

3. Availability

4. Geographic proximity

5. Experience

6. Reliability

MVP method: deterministic rules (gates) plus weighted scoring. **AI assists with explanations and recommendations; it is never the sole decision-maker.** Matching is transparent and auditable: inputs, per-criterion results and final rank are stored with each recommendation. Details in B5.

Explainability — "Why was this volunteer matched?" Each recommendation shows:

- Required skill match (which skills, which certifications)
- Availability match (which shift window)
- Location proximity (distance band, within preferred distance)
- Relevant experience
- Reliability history (or "Not enough history")

Example: "Recommended because the volunteer has the required first-aid certification, is available during the event shift, is within their preferred travel distance, has previous emergency-response experience, and has a consistent attendance history."

Karma is **not** a matching score input. Volunteer preferences (causes) may be used only as a tie-breaker. [P]

AC: Given an event with required skills, when matching runs, then it returns eligible volunteers ranked on skills, certification, availability, proximity, experience and reliability. Given a volunteer lacks a mandatory certification, then they are excluded and can be listed as ineligible with a reason. Given any recommendation, then a readable explanation and per-criterion results are present. Given recommendations, then no assignment exists until a human confirms.

A9.5 Scheduling & Rostering (A basic; B enhanced)

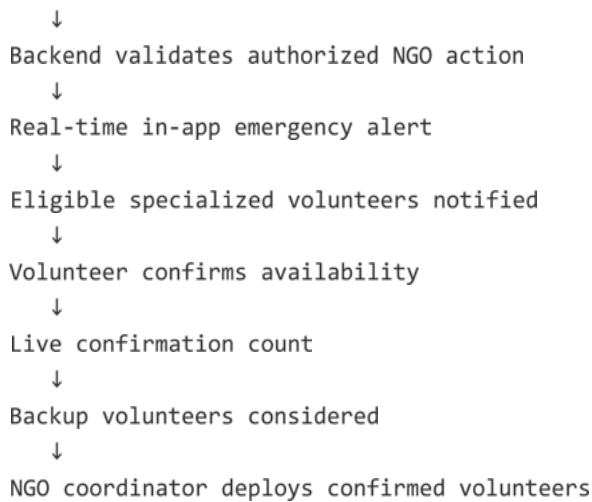
Inputs: event requirements, headcount, skills, shift timings, availability, capacity, recurring schedules, existing assignments, conflicts, workload. Workflow: Event requirements → Availability → Existing commitments → Conflicts → Shift allocation → Backup allocation → Final roster. Algorithm in B6.

Rules: conflicts are shown before finalization [C]; the roster remains Draft until a human finalizes; finalization notifies volunteers; declines suggest backup promotion.

AC: Given requirements and a pool, when generation runs, then a draft roster with primaries (and backups where available) and a conflict report is produced. Given a volunteer already booked in an overlapping shift, then they are not double-booked and the conflict is reported. Given unfilled slots, then they are listed explicitly.

A9.6 Emergency Mobilization (B; basic path as optional demo)

Workflow (see B7 for architecture):



Three distinct things, never conflated:

Concept	Meaning	Owner
Emergency alert	Real-time in-app notice to eligible volunteers (email as secondary fallback)	Triggered by an authorized NGO user
Volunteer confirmation	A volunteer's statement that they are available	Volunteer
Deployment decision	Assigning confirmed volunteers to the response	NGO coordinator/admin

Confirming availability does **not** deploy a volunteer. The Agent **cannot** create or broadcast emergencies; NGO/coordinator makes final decisions. [C]

AC: Given an authorized user creates an emergency, then the backend validates authorization before any alert is sent. Given eligible volunteers, when alerted, then the alert appears in-app in real time and only matched, in-range specialized volunteers receive it. Given volunteers confirm, then the live confirmation count updates. Given confirmations fall short, then backup candidates are proposed for human approval. Given confirmed volunteers, then none is deployed until a coordinator/admin performs the deployment action. Given the Agent suggests an alert, then nothing is sent until a human acts.

A9.7 Reliability (B for advanced analytics; A basic recording)

Reliability measures how consistently a volunteer fulfils assigned commitments and is **not** an absolute judgment of a person. It is one matching factor.

Signals [C]: number of completed assignments, cancellations (with notice), no-shows, recent participation, task completion, participation consistency, punctuality if tracked.

Fairness rules [P]:

- Not based on a simple attendance percentage alone: combines completed assignments, notice-adjusted cancellations, no-shows, recency weighting and task completion.
- Advance cancellations carry little or no penalty; NGO-marked excused absences carry none; no-shows are the strongest signal.

- **Sufficient-history threshold:** below it, display "**Not enough history**" instead of a confident score; matching treats the volunteer neutrally (no penalty for being new). Threshold value: **Must decide (A17)**.
- Display: labelled indicator (e.g., Building history / Reliable / Highly reliable) with underlying counts. Band cut-offs and formula: Must decide. Public visibility: default off.

AC: Given attendance recorded, then reliability inputs update. Given insufficient history, then the UI shows "Not enough history". Given an advance cancellation, then it is not treated as a no-show.

A9.8 Karma / Community Impact (A)

Karma Points are a verified measure of community contribution — a credibility and contribution indicator, **not** a government credential. **No government schemes or benefits; no employment or NGO-position guarantees.** [C]

Inputs: verified attendance, completed volunteer assignments, service hours, participation history, achievements/milestones.

Rules [P]:

- Awarded only after NGO-verified attendance (and task completion where applicable).
- Recorded as an immutable **transaction ledger** (`karma_transactions`: points, reason, event, attendance reference); totals are derived; corrections are compensating transactions.
- Deterministic calculation (not AI-generated). Point values: **Must decide (A17)**. MVP proposal: points per verified hour plus a fixed completion bonus. [P]
- Anti-abuse: no award without verified attendance; duplicate awards for the same attendance blocked; anomalies reviewable by platform admin.
- The Agent can explain Karma but cannot award it.

Uses: demonstrate credibility, verified service profile, recognition, visibility to NGOs, potential eligibility for higher responsibility within participating organizations (no guarantee).

AC: Given verified attendance, when the event closes, then Karma transactions are created and the total updates. Given unverified attendance, then no Karma is awarded. Given duplicate verification, then Karma is awarded once. Given an existing transaction, then it cannot be edited.

A9.9 Independent Volunteer Ecosystem (A)

Independent volunteers can discover public events, view details and NGO indicators, register, participate temporarily, build verified history, discover aligned NGOs, and potentially affiliate later. They access only public events; no access to NGO-only data or other volunteers' contact data.

AC: Given a public event, when an independent volunteer registers, then they appear as a temporary participant without becoming an NGO member. Given verified attendance, then the event appears in their verified history and Karma updates. Given an NGO invites a past participant, then affiliation takes effect only after the volunteer accepts.

A9.10 Event Intelligence Agent (A basic; deeper analysis B)

Name: *SevaSetu Event Intelligence Agent*. (The earlier name "Voltix AI" is retired.) **Principle:** Agent recommends, human decides.

Core responsibilities [C]

- Analyze event requirements
- Analyze previous similar events
- Suggest volunteer preparation
- Identify potential operational risks
- Recommend useful skills/certifications
- Suggest volunteer requirements
- Generate preparation checklists
- Provide event-specific guidance
- Explain relevant historical patterns
- Assist volunteers with event information
- Help independent volunteers understand how to prepare

The Agent MUST NOT [C]

- Create emergencies
- Broadcast emergencies
- Finalize volunteer rosters
- Change attendance records
- Award Karma directly
- Override NGO decisions
- Make irreversible operational decisions

Capability	Input	Processing	Output	Consumer	Handled
Event guidance	Event record, NGO profile, question	Grounded LLM answer	Answer + basis	Volunteers, NGO staff	Validated
Historical analysis	Aggregated past similar events (fill rate, no-show rate, staffing, notes)	Summarize patterns	Summary with "based on N events" caveat	Coordinators	Corrected
Preparation	Requirements, history	Suggest skills, resources, risks	Checklist	Coordinators, volunteers	Not applicable
Staffing/requirements suggestion	Event + pool summary	Suggest requirements/headcounts	Recommendations	Coordinators	Corrected

Emergency preparedness	Event context	Readiness considerations	Suggestions (not predictions)	Coordinators	C d
Volunteer guidance	Event details	Explain role, what to bring	Plain-language answer	Independent volunteers	—

Guardrails: outputs labelled "AI-generated suggestion"; no guaranteed predictions; "insufficient data" fallback instead of fabrication; role- and scope-aware; volunteer-facing sessions never receive other volunteers' personal data. Architecture in B8.

AC: Given a volunteer asks what to bring, then the Agent answers from that event's data, marking uncertain items as suggestions. Given a coordinator asks for staffing/risk suggestions, then output includes its basis and a note that decisions remain theirs. Given insufficient history, then the Agent says so. Given any request to perform a restricted action, then the Agent gives guidance only and points to the human workflow.

A9.11 NGO Transparency & Activity Indicators (A)

Replaces the concept of a simplistic "NGO Rating." SevaSetu does **not** declare an NGO good or bad, does not definitively identify fraudulent or fake NGOs unless independently verified evidence exists, and is **not** a legal certifier. It provides transparency and activity indicators to help volunteers and donors make more informed decisions.

Profile content: organization information; causes/focus areas; events conducted; event completion; volunteer participation; activity history; transparency indicators (completeness of information provided); verified platform activity; participation consistency; publicly available impact information; fund information where provided and verifiable.

Data sources [P]: *platform-derived* (from Supabase event, roster and attendance records) and *NGO-provided* (labelled "provided by NGO", with a verification status where applicable). NGO verification approach: **Must decide (A17)**; advanced verification is [F].

Display [P]: a breakdown of separate indicators (activity, completion, participation, consistency, information completeness), not one opaque score, with this notice: *"Platform-derived indicators based on activity and information available on SevaSetu. They are not a certification, legal verification, or guarantee of an NGO's legitimacy or of how donations will be used."*

AC: Given an NGO completes events with recorded attendance, then its activity indicators update. Given NGO-provided fund information, then it is labelled "provided by NGO" with its verification status. Given any indicator is displayed, then the notice is displayed. Given any profile page, then no wording claims fraud, certification or guaranteed legitimacy.

A9.12 Analytics, Retention & Donor Experience (B)

- Tracked: service hours, milestones, recognition, participation frequency, engagement patterns, retention, potential burnout indicators (signals, not diagnoses; thresholds TBD).
- Interventions: recognition, milestones, suggested breaks, better workload distribution, re-engagement opportunities.
- Basic NGO dashboard (Tier A support for the demo's final step): events, participation, attendance, hours, repeat volunteers.

- Donor/public: browse NGOs, view profile and indicators, no login required. Payments/donation processing are not in scope (TBD).

A9.13 Notifications & Maps (A basic; B advanced)

- Notification types [C]: event invitations, assignment, shift reminders, emergency alerts, volunteer confirmations, schedule changes, event updates, participation confirmations.
- Channels: **in-app first** (real-time for emergencies), **email as secondary/fallback** via Email API; n8n for workflow automation. No SMS/WhatsApp assumed (TBD/future). [C/P]
- Maps API [C]: volunteer proximity, event location, distance-aware matching, location-based discovery, emergency volunteer discovery; privacy-conscious handling (B11).

A10. User Stories

Priority: **M** Must / **S** Should / **C** Could / **F** Future. Tier shown as A/B/C.

NGO Admin

ID	Story	Pri	Tier
A1	As an NGO admin, I want to create and edit our NGO profile, so that volunteers and donors understand our work.	M	A
A2	As an NGO admin, I want to create events with skill, certification and headcount requirements, so that the right volunteers can be found.	M	A
A3	As an NGO admin, I want to publish events publicly, so that independent volunteers discover us.	M	A
A4	As an NGO admin, I want to add transparency information, so that donors and volunteers can see our activity.	M	A
A5	As an NGO admin, I want to create an emergency and alert eligible specialized volunteers, so that we can mobilize quickly.	S	B
A6	As an NGO admin, I want to manage coordinators, so that day-to-day work can be delegated.	S	A/B
A7	As an NGO admin, I want an analytics dashboard, so that I can improve engagement and retention.	S	B
A8	As an NGO admin, I want to invite past participants to affiliate, so that we build longer-term relationships.	S	B

NGO Coordinator

ID	Story	Pri	Tier
C1	As a coordinator, I want ranked, explainable volunteer recommendations, so that I can confirm assignments confidently.	M	A

C2	As a coordinator, I want an auto-generated roster with conflicts flagged, so that I avoid double-booking and gaps.	M	A
C3	As a coordinator, I want to mark attendance and task completion, so that records and Karma are accurate.	M	A
C4	As a coordinator, I want to ask the Agent for preparation and risk suggestions, so that I plan better.	M	A
C5	As a coordinator, I want backup volunteers on the roster, so that no-shows don't break the event.	S	A/B
C6	As a coordinator, I want to see the live confirmation count in an emergency and deploy confirmed volunteers, so that I stay in control.	S	B
C7	As a coordinator, I want to see burnout indicators, so that I can balance workload.	C	B

Independent Volunteer

ID	Story	Pri	Tier
I1	As an independent volunteer, I want to browse public events by cause, date and distance, so that I can find where to help.	M	A
I2	As an independent volunteer, I want to review an NGO's transparency and activity profile, so that I choose organizations wisely.	M	A
I3	As an independent volunteer, I want to register without joining the NGO, so that I can participate temporarily.	M	A
I4	As an independent volunteer, I want verified participation to build my Karma and history, so that I gain credibility.	M	A
I5	As an independent volunteer, I want the Agent to explain what to bring and what the role involves, so that I'm prepared.	S	A
I6	As an independent volunteer, I want suggestions for NGOs aligned with my causes, so that I build connections.	S	B
I7	As an independent volunteer, I want control over what location/profile data is visible, so that my privacy is protected.	M	A

Affiliated Volunteer

ID	Story	Pri	Tier
V1	As an affiliated volunteer, I want to maintain skills, certifications, availability and location, so that I'm matched properly.	M	A
V2	As an affiliated volunteer, I want to accept/decline assignments, so that I can manage	M	A

my time.

V3	As an affiliated volunteer, I want to see hours, Karma, reliability summary and achievements, so that I see my impact.	M	A
V4	As an affiliated volunteer, I want to receive and confirm emergency alerts in real time, so that I can help in a crisis.	S	B
V5	As an affiliated volunteer, I want suggested breaks when heavily engaged, so that I avoid burnout.	C	B

Donor / Public User

ID	Story	Pri	Tier
D1	As a donor, I want to browse NGOs by cause, so that I find organizations I care about.	M	A
D2	As a donor, I want to see activity history and transparency indicators, so that I can make an informed decision.	M	A
D3	As a donor, I want it made clear the indicators are platform-derived, not a certification, so that I don't over-trust them.	M	A

Event Intelligence Agent

ID	Story	Pri	Tier
G1	As the Agent, I want scoped event details and permitted historical data, so that I can give grounded guidance.	M	A
G2	As the Agent, I want to recommend skills, resources, requirements and risks, so that NGOs prepare better.	M	A
G3	As the Agent, I want to label outputs as suggestions with their basis, so that humans keep decision authority.	M	A
G4	As the Agent, I want to summarize recurring issues from similar events, so that coordinators avoid repeats.	S	A/B
G5	As the Agent, I want to suggest emergency-readiness considerations (without triggering anything), so that teams plan ahead.	S	B

Platform Administrator

ID	Story	Pri	Tier
P1	As a platform admin, I want to moderate accounts, so that the platform stays trustworthy.	S	B
P2	As a platform admin, I want aggregate platform metrics, so that I can track adoption.	S	B

P3 As a platform admin, I want an NGO verification workflow, so that profiles carry stronger assurance. F C

A11. MVP Scope

SevaSetu is designed as a scalable platform, but the hackathon MVP intentionally implements a focused subset of the complete vision.

Level	Contents
A. Demo-Critical MVP	Authentication & roles; volunteer profile; NGO profile; event creation; public event discovery; volunteer registration; multi-criteria matching with explanations; automated/basic roster generation; attendance/check-in; Karma and verified history; NGO Transparency & Activity Profile; basic Event Intelligence Agent
B. Secondary / Demo-Optional	Advanced reliability analytics; advanced retention analytics; burnout indicators; emergency mobilization enhancements; advanced notification workflows; advanced analytics dashboards
C. Future	Per A12

Permitted MVP simplifications [P]: seeded demo data (NGOs, volunteers, past events for the Agent); fixed matching weights; fixed Karma values; greedy rostering; button-based check-in; in-app notifications with email fallback; single primary LLM with a fallback.

A12. Future Scope [F]

Advanced capabilities retained in the product vision but not required for the hackathon MVP:

- Larger-scale NGO networks and multi-region deployment
- Advanced predictive analytics
- Advanced burnout detection
- More sophisticated emergency coordination (specialized group management at scale, automated escalation rules with human approval, geo-verified check-in)
- Additional communication channels (e.g., SMS/WhatsApp)
- Advanced NGO verification mechanisms (independently verified evidence)
- Learned/ML-tuned matching weights (with explainability preserved)
- More integrations
- Billing implementation and subscription management

Not returning to scope: government schemes/benefits tied to Karma. NGO transparency is **core**, not future.

A13. Success Metrics

No numeric targets are set; any target should be labelled a "proposed benchmark" once baseline data exists.

Group	Metrics
Volunteer	Registration completion rate; event discovery (views → event detail); registration conversion (detail → registered); attendance; service hours; repeat participation
NGO	Events created; volunteer matching success (recommended → confirmed); roster completion (filled/required slots); event completion; volunteer retention; operational time saved (e.g., time from event creation to finalized roster, compared with baseline NGO process)
Platform	Match success rate (recommended → confirmed → attended); attendance rate; volunteer retention; event fulfillment (events meeting required headcount); emergency response confirmation rate; NGO activity; community participation (unique participants per period)
Emergency	Alert reach (eligible volunteers who saw the alert); confirmation rate; deployment completion; time to mobilization
Agent (suggested) [P]	Sessions; share of outputs marked helpful; "insufficient data" rate

A14. Competitive & Research Insights

Qualitative, structural gap analysis only; no competitor statistics are claimed.

Existing approach	Limitation	SevaSetu improvement
Basic volunteer opportunity portals	Mostly opportunity discovery	Adds matching, scheduling, attendance and impact tracking
Manual NGO coordination	High operational effort	Automated matching and roster generation
Generic volunteer ratings	Weak contribution verification	Verified Karma and service history, kept distinct from reliability
Static NGO profiles	Limited visibility into actual activity	NGO Transparency & Activity Indicators from real platform activity
Generic AI assistants	Limited operational context	Event-specific Intelligence Agent grounded in event and permitted history data

Validation note: the above reflects the team's product thesis; direct NGO and volunteer interviews would be the way to validate it (future activity).

A15. Risks

Risk	Impact	Mitigation
------	--------	------------

Emergency alerts not seen or acted upon in time	Delayed mobilization	Real-time in-app alerts first; email only as fallback; additional channels as future scope; human-controlled deployment
AI hallucination/overclaiming	Wrong guidance	Grounded context, "insufficient data" fallback, labelled suggestions, Agent has no write actions
Cold start (no volunteers/NGOs/history)	Weak matching and Agent output	Seeded demo data; neutral treatment of new volunteers
Location privacy leakage	Safety/trust	Coarse sharing, server-side distance, role-scoped access
NGO misrepresentation of information	Trust loss	"Provided by NGO" labels, verification status, careful language, moderation
Karma gaming	Credibility loss	NGO-verified attendance, append-only ledger, anomaly review
Reliability perceived as unfair	Volunteer churn	Non-punitive rules, excused absences, "Not enough history" state, one factor only
Karma/reliability conflated	Confusing UX, unfair matching	Distinct UI and data, Karma excluded from matching score
Two-datastore complexity	Integration bugs	Strict ownership rules, ID references, backend-only Firebase access
Scope too broad for hackathon	Incomplete demo	Tiered MVP, seeded data, defined cut order
LLM/Maps/Email usage costs	Margin pressure on the ₹250 per NGO per month model	Usage limits, cost tracking, pricing validation
Regulatory/data-protection	Compliance gaps	Early review (TBD), data minimization

A16. Business Model

The primary focus of SevaSetu is social impact; the subscription exists to sustain the platform.

NGO subscription: ₹250 per NGO per month. This is the current proposed pricing assumption, not a validated market price; pricing validation with NGOs is a future business-validation activity.

Value provided to NGOs through the subscription

Capability	Benefit
Volunteer recruitment	Reach independent and affiliated volunteers

Volunteer matching	Explainable recommendations for each event
Event promotion	Public event discovery
Scheduling and roster generation	Fewer gaps and conflicts, less manual effort
Attendance and participation tracking	Verified records
NGO transparency profile	Public activity indicators that build trust
Volunteer communication	In-app and email notifications
Analytics and operational insights	Participation, retention and hours
Event Intelligence Agent assistance	Preparation and risk guidance

Volunteer and donor/public access to browse and participate is assumed to carry no subscription (not specified in the brief; TBD). Billing implementation is not part of the hackathon MVP.

A17. Critical Decisions

Must Decide Before MVP Development

Each has a proposed default so development is not blocked; the team should confirm or change it.

#	Decision	Why it blocks development	Proposed default [P]
1	Matching weights	Drives ranking code and explanations	Gates first; then weights in priority order: skills/certs 35, availability 20, proximity 20, experience 15, reliability 10 (sum 100); causes as tie-breaker; weights in a config object
2	Karma calculation	Ledger rules and UI	Points per verified service hour + fixed bonus per completed assignment; milestone thresholds as config; Karma not used in matching
3	Reliability calculation	Display and matching input	Recency-weighted combination of completed assignments, notice-adjusted cancellations, no-shows, task completion; insufficient-history threshold as config (e.g., a small number of completed assignments); label bands instead of raw percentage
4	Certification verification approach	Eligibility gates	Self-declared with optional "NGO-verified" flag; mandatory-cert roles may require the verified flag per event
5	Emergency permissions	Authorization rules	NGO admin creates and alerts; coordinators view, manage confirmations and deploy
6	Emergency	Core of B7	Alert → volunteer Accept/Decline in-app → live count →

		confirmation workflow	coordinator deploys; confirmation ≠ deployment
7	NGO verification approach	What the profile can claim	MVP: no legal verification; "provided by NGO" vs "platform-derived" labels only; verification status field kept for later
8	NGO transparency indicators	Profile content	Activity, completion, participation, consistency, information completeness as separate Low/Medium/High-style indicators with the disclaimer
9	Maps provider	Geocoding and distance code	Choose one provider before coding the location layer; store lat/long in Supabase (PostGIS) so the provider is swappable
10	LLM routing	Agent service design	One primary LLM, one fallback; Hugging Face only if embeddings/classification are needed
11	Python Agent service hosting	Deployment of the intelligence path	A host suited to a long-running/ML-friendly service (not forced into serverless); provider TBD but responsibility separated (B13)
12	Firebase real-time usage	Emergency alert/live-count mechanism	Firebase used for ephemeral real-time alert and live-count state and for agent state; Supabase remains the record of truth

Can Remain TBD / Future Scope

Burnout thresholds; workload caps and backup ratios (use simple config defaults); reminder timing offsets; agent context TTL; data retention and account-deletion policy; legal/compliance review (e.g., Indian data-protection requirements); localization; whether minors may volunteer; donation processing; billing implementation; platform admin tooling depth; advanced verification mechanisms; additional channels.

A18. Hackathon Demonstration Flow

Primary demo (single end-to-end story, ≈7 min):

1. **Volunteer creates/opens profile** — skills, certifications, availability, location, interests.
2. **NGO creates an event** — public, with capacity.
3. **Event requirements are defined** — roles, required skills/certifications, shifts.
4. **Matching engine recommends volunteers** — ranked list; one ineligible volunteer shown with reason.
5. **Matching explanation is shown** — "Why was this volunteer matched?"
6. **Roster is generated** — conflicts flagged, backups shown, coordinator finalizes.
7. **Volunteer confirms participation.**
8. **Attendance is recorded** — check-in and NGO verification.
9. **Karma is updated** — ledger entry and history updated; reliability counts update (Karma and reliability shown as separate things).

- 10. **NGO transparency profile is displayed** — indicators with the not-a-certification notice.
- 11. **Event Intelligence Agent analyzes the event** — preparation checklist, risks, skill suggestions, historical pattern with basis; coordinator reminded decisions are theirs.
- 12. **Dashboard shows the resulting impact** — participation, hours, attendance.

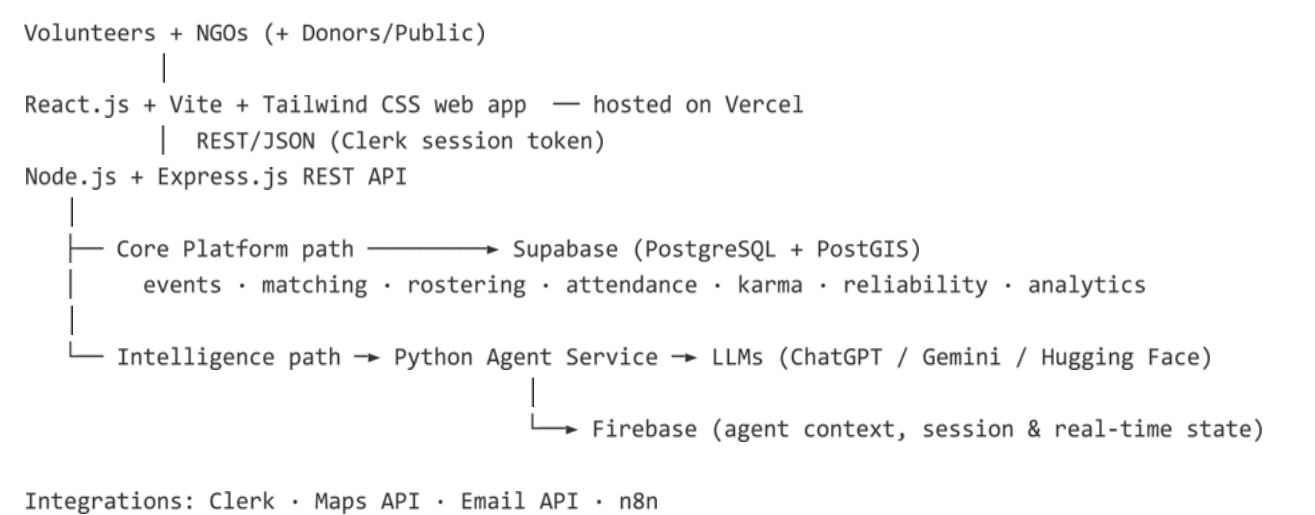
This is the primary demonstration because it shows the most SevaSetu differentiators (matching, independent ecosystem, Karma, transparency, Agent, integrated operations) in one coherent flow.

Optional secondary scenario — Emergency mobilization (≈1.5 min): authorized NGO user creates an emergency → real-time in-app alert to eligible specialized volunteers → confirmations with live count → backup proposed → coordinator deploys. Emphasize: alert, confirmation and deployment are separate; the Agent suggests nothing automatically.

Demo safeguards: pre-seed data; pre-test real-time alert path; keep a recorded fallback for emergency and Agent segments.

PART B — TECHNICAL IMPLEMENTATION SPECIFICATION

B1. System Architecture



Conceptual deployment chain [C]: React/Vite Frontend → Vercel → Node.js/Express API → Python Agent Service → LLM.

Event-centred view of the architecture: every Express module (matching, rostering, emergency, attendance, karma, analytics, agent context builder) takes an `event_id` (or `emergency_id`, which is an event-linked urgent request) as its primary key and reads/writes records related to that event. The Agent receives context built from an event; it never writes back to core tables.

Component	Responsibility
React/Vite/Tailwind app	UI for all roles, discovery, dashboards, Agent chat, real-time alert UI
Express API	Auth; deterministic business logic (matching, rostering, attendance, Karma

Express API	Admin, deterministic business logic (matching, rostering, attendance, karma, reliability, permission checks); orchestration of Agent, notifications, Maps
Supabase	Persistent relational source of truth
Python Agent Service	Prompting, model routing, retrieval over provided context, output validation
Firebase	Agent sessions/context/state; ephemeral real-time state
Clerk	Identity, sessions, roles
Maps API	Geocoding, maps, distance
Email API + n8n	Secondary notification delivery and workflow automation
Vercel	Frontend and Express hosting

B2. Technology Stack (exactly as specified) [C]

Layer	Technology
Frontend	HTML, React.js, Vite, Tailwind CSS
Backend	Node.js, Express.js, REST API
Core database	Supabase
Agent data/state	Firebase
AI & ML	Python, ChatGPT, Gemini, Hugging Face
Services	Clerk, Maps API, Email API, n8n
Deployment	Vercel (frontend and API); Python Agent Service on a suitable environment (see B13)

Development/design tools (GitHub, GitHub Copilot, Cursor, Lovable, Stitch, Impeccable, Motion Dev, Antigravity, Taste Skill) are development aids only and are **not** runtime architecture components.

B3. Database Architecture

B3.1 Responsibilities

Supabase — Core Platform Source of Truth [C]

Domain	Contents
Identity & profiles	Volunteer profiles, NGO profiles, memberships, affiliations
Capabilities	Skills, certifications, availability
Events (central)	Events, event requirements, shifts, volunteer registrations (applications)

Events (calendar, events, event requirements, times, volunteer registrations (applications, Operations	Assignments, rosters, attendance, tasks, emergency requests and responses
Contribution	Karma transactions, achievements, reliability data
Transparency	NGO transparency data, indicators
Insight	Core analytics; relationships between platform entities
Communication records	Notifications

Firestore — Agent Context and Real-Time State [C]

Domain	Contents
Agent conversations	Conversation/context state, messages
Agent sessions	Session state and activity
Short-lived operational context	Scoped context snapshots with expiry
Real-time agent state	In-progress steps, flags
Real-time notification/event state (if required)	Ephemeral alert delivery and live confirmation-count state, derived from and never replacing the Supabase records

B3.2 Why two data systems (stated plainly)

A single database could technically support both workloads. SevaSetu intentionally separates core transactional platform data from Agent-specific context and real-time state to maintain clearer responsibility boundaries, independent scaling, and simpler Agent-state management.

Two databases are **not** technically mandatory; the cost is extra integration complexity (two systems, ID mapping, consistency), which the rules below contain.

B3.3 Non-overlap rules

1. **No duplication of master data in Firestore.** Firestore documents hold IDs referencing Supabase plus short-lived, minimal snapshots. [C]
2. Agent context is built on demand by Express from Supabase, scoped to the caller's permissions, and expires (TTL TBD).
3. Anything operationally authoritative — registrations, rosters, attendance, Karma, reliability, emergency confirmations and deployments — is written to **Supabase first**. Firestore real-time state, where used, is a derived view and can be rebuilt from Supabase.
4. Agent outputs become platform records only when a human performs the action through normal workflows.
5. Firestore is accessed through the backend (Admin SDK) for writes; direct client writes are disabled by default; clients may subscribe to scoped real-time channels read-only. [P]

B4. Data Model

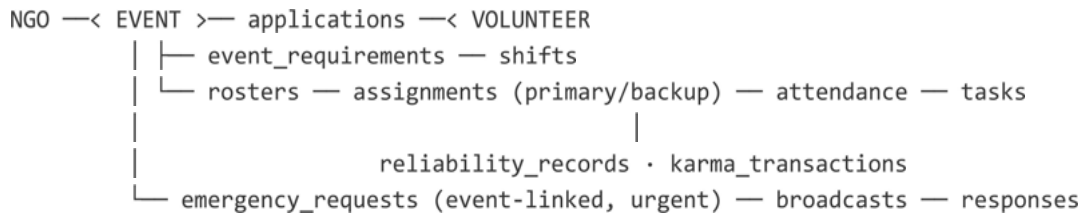


Table	Key columns	Relationships / ownership
users	id, clerk_user_id, email, role, status	1–1 volunteer_profile (if volunteer)
volunteer_profiles	user_id, name, bio, home_geo, max_distance_km, causes[], privacy settings	Volunteer-owned
skills	id, name, category	Master list
volunteer_skills	volunteer_id, skill_id, level	M–N
certifications	id, volunteer_id, name, issuer, expiry, doc_url, verified_by_ngo_id?	Volunteer-owned
availability	id, volunteer_id, weekday/time window or date range, recurring	Volunteer-owned
ngos	id, name, description, causes[], region, geo, contact	NGO-owned
ngo_members	ngo_id, user_id, role	M–N
ngo_affiliations	ngo_id, volunteer_id, status (invited/active/ended)	M–N
events	id, ngo_id, title, description, cause, geo, start/end, visibility, capacity, status	Central entity, NGO-owned
event_requirements	id, event_id, role, headcount, required_skill_ids[], required_cert_names[], min_experience	Event-owned
shifts	id, event_id, requirement_id?, start, end, headcount	Event-owned
applications	id, event_id, volunteer_id, status (interested/accepted/waitlisted/rejected)	Registrations
match_runs	id, event_id, run_by, config_snapshot (weights),	Audit of each

	created_at	matching run
match_results	id, match_run_id, volunteer_id, eligible, rank, criteria_results (jsonb), explanation	Auditable explanations
rosters	id, event_id, status (draft/final), generated_at, conflict_report (jsonb)	Event-owned
assignments	id, event_id, shift_id, volunteer_id, type (primary/backup), status (proposed/confirmed/declined/deployed), match_result_id?	
attendance	id, assignment_id, volunteer_id, checkin_at, checkout_at?, status (attended/no_show/excused/cancelled), verified_by	NGO-verified
tasks	id, event_id, assignment_id?, description, status, completed_at, verified_by	
reliability_records	id, volunteer_id, event_id, assignment_id, outcome, notice_hours?, created_at	Append-only; indicator derived
karma_transactions	id, volunteer_id, event_id, attendance_id, points, reason, created_at	Append-only ledger
achievements / volunteer_achievements	criteria config; volunteer_id, achievement_id, earned_at	
emergency_requests	id, ngo_id, event_id?, title, description, geo, required_skills, headcount, urgency, status, created_by	Urgent event-linked request
emergency_broadcasts	id, emergency_id, sent_by, sent_at, target_filter (jsonb), recipient_count	Alert record
emergency_responses	id, broadcast_id, volunteer_id, response (confirmed/declined), responded_at, is_backup, deployed_at?, deployed_by?, checked_in_at?	Confirmation vs deployment are separate columns
ngo_transparency_records	id, ngo_id, type, content, source (platform/ngo), verification_status	
ngo_indicators	ngo_id, indicator values, computed_at	Derived from event/attendance data
notifications	id, user_id, type, payload, channel (in_app/email), status, created_at	

Ownership rules: volunteers own profile/availability/certifications; NGOs own events and event data; attendance, reliability and Karma are system-of-record data written only by backend logic after a verified human action.

B4.2 Firebase collections [P]

Collection	Contents	Notes
agentSessions/{sessionId}	userId (Clerk), role, ngold?, eventId?, mode (volunteer/ngo), createdAt, status	IDs only, no master data
agentSessions/{id}/messages/{msgId}	role, content, timestamp	Conversation state
agentContext/{sessionId}	Scoped snapshot (event summary, aggregated similar-event stats), source IDs, built_at, ttl	Rebuilt from Supabase on demand
agentActivity/{activityId}	sessionId, action type, model, latency, status	Observability
agentOutputs/{outputId}	sessionId, type (guidance/preparation/staffing/risk), content, basis (source IDs), disclaimer flag, humanAction (none/accepted/dismissed)	Suggestions; accepted items re-entered by humans in Supabase
agentState/{sessionId}	Conversation state, pending steps, flags	Real-time agent state
realtime/emergencies/{emergencyId} (optional, A17 #12)	Alert status, live confirmed count, per-volunteer alert-seen/confirmed flags (IDs)	Ephemeral, derived from Supabase; non-authoritative

B5. Matching Architecture

Pipeline (Node/Express, deterministic, POST /events/:id/matches):

1. **Eligibility (gate).** Volunteer active, opted in to matching, not already in a conflicting assignment, within their own max travel distance, event visibility permits (independent volunteers only for public events).
2. **Required skills/certifications (gate + score).** Mandatory skills/certifications missing or expired → ineligible (listed with reason). Optional skills contribute to score.
3. **Availability (gate + score).** Must cover the shift; scored by fit (full coverage, existing workload).
4. **Geographic proximity (score).** Server-side distance from volunteer's stored location to event; distance band.
5. **Experience (score).** Relevant verified participation (same cause/role), from event history.

6. **Reliability (score, lowest weight).** From the reliability indicator; "Not enough history" → neutral value, no penalty.

Scoring [P]: weighted sum of normalized criterion results using the config in A17 #1 (skills/certs 35, availability 20, proximity 20, experience 15, reliability 10). Weights are a config object stored with each `match_run` for auditability. **Karma is not an input.** Tie-breaker: cause preference, then earlier registration.

Explainability output (stored in `match_results`): per-criterion result (met/partial/not met + value), distance band, reliability label, and a generated sentence. The explanation sentence is produced from the stored criterion results by a template; the Agent/LLM may *rephrase* it but cannot change rank or eligibility. [P]

Audit: `match_runs` + `match_results` record who ran matching, with which weights, and what each volunteer's results were. Assignments reference the result that led to them.

Human gate: recommendations become assignments only through coordinator confirmation (`POST /events/:id/matches/confirm`).

B6. Scheduling Architecture

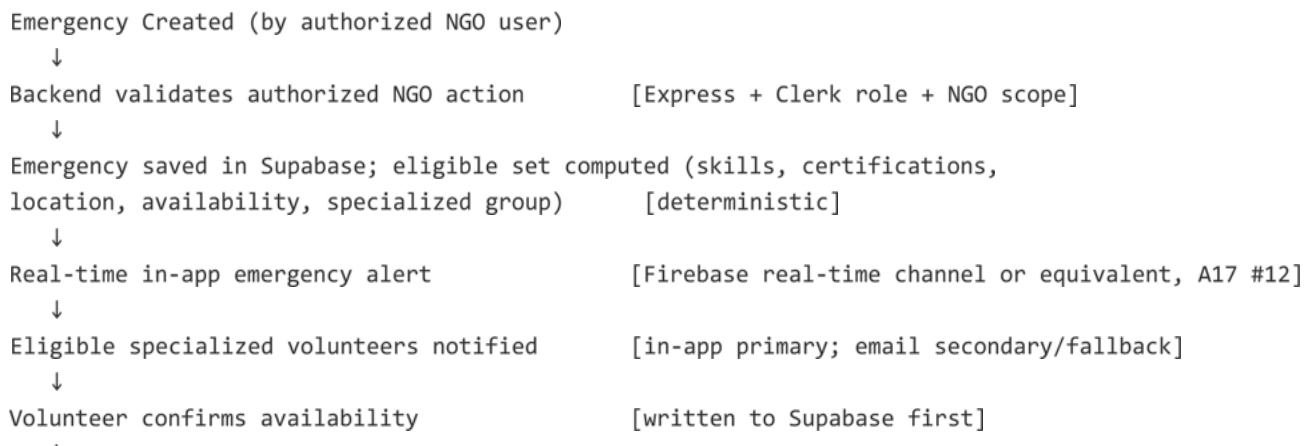
Pipeline (`POST /events/:id/roster/generate`) [P]:

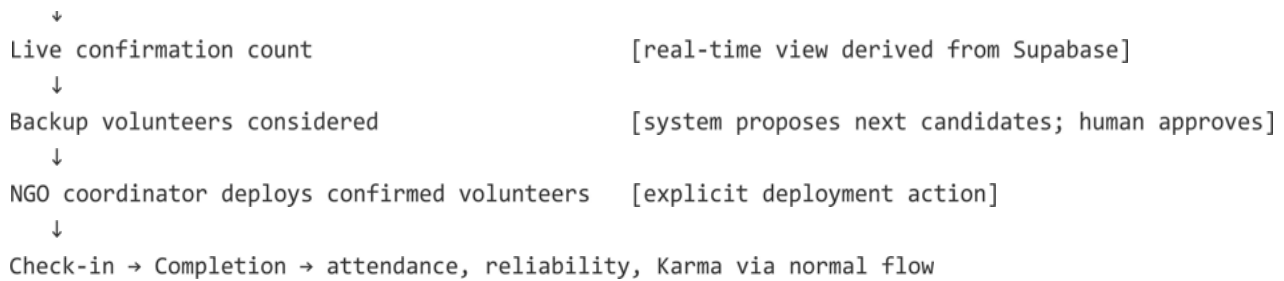
1. Collect shifts and requirements, plus confirmed/registered candidates and match results.
2. For each shift/role, allocate the highest-ranked candidate who is available and not in an overlapping assignment (any event).
3. Apply a workload cap per volunteer (config; default TBD) and prefer less-loaded volunteers on ties.
4. Allocate backups after primaries (count per role as config; proposal $\approx$ 1 backup per 3–5 primaries).
5. Produce a conflict report: unfilled slots, overlaps, over-capacity, expired certifications.
6. Save as Draft roster; coordinator edits; **finalize is a human action** (`/roster/finalize`), which creates notifications and flips assignments to awaiting confirmation.

Recurring availability is expanded into concrete windows for the event dates. Greedy allocation (not an optimization solver) is intentional for MVP simplicity and explainability; optimization is future scope.

B7. Emergency Architecture

Authority and sequence





Design rules

1. **Email is not the primary emergency mechanism.** In-app real-time alert is primary; email is secondary notification/fallback (e.g., for volunteers who haven't opened the app after a configured delay — value TBD).
2. **Source of truth:** `emergency_requests`, `emergency_broadcasts`, `emergency_responses` live in Supabase. The real-time layer only mirrors alert state and counts for speed; if it fails, polling the API still works.
3. **Three separate states** per volunteer: *alerted* → *confirmed (available)* → *deployed*. Only the deployment action by a coordinator/admin sets deployed.
4. **Specialized volunteer groups:** dynamic groups derived from skills/certifications (e.g., medical, rescue); persistent group management is future scope.
5. **Location-aware targeting:** distance computed server-side; coordinates not sent to clients.
6. **Backup logic:** if confirmations < required headcount after a configurable wait (TBD), propose next-ranked or wider-radius candidates for human approval.
7. **Agent boundary:** the Agent may provide readiness guidance beforehand or suggest requirements when asked; it has no endpoint or credential able to create, alert, or deploy. All emergency endpoints require a human Clerk session with an authorized role.
8. **Idempotent confirmation endpoints** and an audit log of who created, alerted and deployed.
9. **Check-in method:** button for MVP; QR/geo-verification future.

B8. Agent Architecture

Principle: Agent recommends, human decides.

Request flow [P]

1. Client → POST `/agent/sessions/:id/messages` with Clerk token.
2. Express verifies identity/role/NGO scope and builds a minimal context object from Supabase (event details, requirements, aggregated similar-event stats, NGO profile summary — never other volunteers' personal data for volunteer mode).
3. Express calls the **Python Agent Service** with `{role, mode, question, context, constraints}` (service-to-service auth).
4. Python service retrieves/organizes context, builds a prompt grounded **only** on supplied context, calls the selected LLM, requests structured output (answer, basis source IDs, uncertainty note).
5. Output validated (guardrails, no private data, disclaimer attached).

- 6. Session, context snapshot, output and activity written to Firebase.
- 7. Response returned to the client; any action the user wants to take happens through normal platform UI/endpoints.

Capability boundaries

- *Allowed*: analyze requirements and similar past events; suggest preparation, skills/certifications, volunteer requirements, checklists, risks; explain historical patterns; answer event questions; help independent volunteers prepare.
- *Prohibited (enforced architecturally, not only by prompt)*: create/broadcast emergencies, finalize rosters, change attendance, award Karma, override NGO decisions, make irreversible decisions. The Agent Service holds **no write credentials** to Supabase and exposes no tools that call write endpoints. [P]

AI vs deterministic logic

Use AI	Use deterministic logic
Event guidance and preparation	Authentication and permission checks
Historical event analysis	Eligibility checks and matching rank
Operational recommendations	Final roster constraints
Volunteer assistance, natural-language interaction	Attendance recording
Rephrasing match explanations	Karma and reliability calculation rules
	Critical emergency actions

Models [P]: one primary LLM (ChatGPT or Gemini) with the other as fallback; Hugging Face models optionally for embeddings (similar-event retrieval) or classification; Python for orchestration. Routing rules: Must decide (A17 #10).

Safety: probabilistic wording; "insufficient data" fallback; prompt-injection defence (NGO- or user-provided text treated as data, no write tools); per-user rate limits; usage logging; consider minimizing personal data sent to third-party LLM providers.

Example specification (historical analysis): *Input* — aggregated stats of the NGO's past similar events (headcount, fill rate, no-show rate, notes). *Processing* — summarize patterns and recurring issues. *Output* — structured summary with number of events considered and caveats. *Consumer* — coordinator. *Stored* — Firebase agentOutputs . *Human decision* — coordinator decides whether to change plans; adopted changes are made manually in Supabase via the normal UI.

B9. API Design

REST/JSON under /api/v1 . Protected endpoints require a valid Clerk token plus role/NGO-scope checks. [P] (Schemas to be defined during build; no unnecessary endpoints.)

Area	Endpoint	Purpose	Access	Tier
Auth/User	GET /me , PATCH /me	Current user,	Authenticated	A

		basics		
	POST /webhooks/clerk	User sync (signed)	Clerk	A
Volunteers	GET/PUT /volunteers/me/profile	Own profile	Volunteer	A
	`PUT /volunteers/me/skills	certifications	availability`	Sub-resources
	`GET /volunteers/me/history	karma	achievements	reliability`
NGOs	POST /ngos, GET/PATCH /ngos/:id	NGO CRUD	NGO Admin	A
	GET/POST/DELETE /ngos/:id/members	Members	NGO Admin	A/B
	GET /ngos/:id/volunteers; POST /ngos/:id/affiliation-invites	Affiliates, invites	NGO staff	B
Events (central)	POST /events, PATCH /events/:id, POST /events/:id/publish	Manage	NGO staff	A
	GET /events (cause/date/geo filters), GET /events/:id	Discovery, details	Public/scoped	A
	`POST /events/:id/requirements	shifts`	Define needs	NGO staff
	POST /events/:id/register	Registration	Volunteer	A
Matching	POST /events/:id/matches	Run matching (stores run/results)	NGO staff	A
	GET /events/:id/matches/:runId	View results & explanations	NGO staff	A
	POST /events/:id/matches/confirm	Confirm → assignments	NGO staff	A
Scheduling	POST /events/:id/roster/generate, GET/PATCH /events/:id/roster, POST /events/:id/roster/finalize	Roster	NGO staff	A
	POST /assignments/:id/respond	Accept/decline	Volunteer	A
Attendance	POST /events/:id/attendance	Check-in / verify	Volunteer (check-in) / NGO staff (verify)	A
	POST	Task	NGO staff	A/B

		/events/:id/tasks/:taskId/complete	completion		
		POST /events/:id/close	Close → reliability, Karma, analytics	NGO staff	A
Karma		GET /karma/:volunteerId	Own/permitted view	Scoped	A
Transparency		GET /ngos/:id/transparency	Public profile & indicators	Public	A
		PUT /ngos/:id/transparency	NGO-provided records	NGO Admin	A
Agent		POST /agent/sessions, POST /agent/sessions/:id/messages, GET /agent/sessions/:id	Agent interaction	Authenticated	A
Analytics		GET /ngos/:id/analytics; GET /admin/analytics	Dashboards	NGO staff / Platform Admin	A basic / B
Emergency		POST /emergencies	Create (validates authorization)	NGO Admin	B
		POST /emergencies/:id/alert	Send real-time alert	NGO Admin	B
		POST /emergencies/:id/respond	Volunteer confirm/decline	Volunteer	B
		GET /emergencies/:id/status	Live counts (also real-time channel)	NGO staff	B
		POST /emergencies/:id/backups	Approve backups	NGO staff	B
		POST /emergencies/:id/deploy	Deploy confirmed volunteers	NGO staff	B
		POST /emergencies/:id/checkin	Check-in	Volunteer	B
Notifications		GET /notifications, PATCH /notifications/:id/read	Inbox	Authenticated	A/B

B10. Authentication & Authorization

- **Authentication (Clerk):** identity, sessions, role claims. Express validates the Clerk JWT on each protected request.
- **Authorization:** role + NGO-scope checks in Express middleware per endpoint; Supabase RLS as a second layer; the Python Agent Service only accepts requests from the Express backend (service credential) and carries the caller's role/scope in the request context.
- **Boundaries:** per matrix in A6. Emergency creation/alert/deploy endpoints are human-only; no service credential used by the Agent can reach them.
- **Role model:** a user has a platform role; NGO roles come from `ngo_members`. Independent vs affiliated status is derived from `ngo_affiliations`.

B11. Security & Privacy

Security [C]: HTTPS; Clerk authentication; role/NGO-scoped authorization; secrets in environment variables (never client-side); service keys server-side only; input validation; rate limiting; audit logging for sensitive actions (matching runs, roster finalization, attendance verification, Karma, emergency create/alert/deploy) [P]; signed webhooks; provider-default encryption at rest and in transit.

Privacy [C]

- **Data minimization:** collect only what matching, rostering and verification need.
- **Location protection:** precise coordinates restricted; NGOs see distance bands only; other volunteers never see them; public pages show no personal location; distance computed server-side; raw coordinates never sent to clients.
- **Privacy controls:** volunteer chooses whether profile is visible to NGOs for recruitment, whether Karma/history is publicly visible, and notification preferences.
- No public exposure of contact details, address, precise location, or detailed history; any public volunteer view shows only opt-in fields (TBD).
- **Agent privacy:** minimal, permission-scoped context; no other volunteers' personal data in volunteer-facing sessions; consider PII minimization before sending data to external LLM providers.
- Retention/deletion policy and legal compliance review: TBD.

B12. Scalability, Performance & Monitoring

Scalability [C]: supports more NGOs, volunteers, events and regions: stateless API; indexed geo (PostGIS) and availability queries; per-event partitioning of work (matching/roster computed per event); Agent service scalable independently of the core API.

Availability [C]: core coordination (event view, registration, confirmation, check-in) must remain reliable during high-demand events; Agent/LLM outage degrades only Agent features; real-time layer failure falls back to API polling.

Performance [C]: reasonable response times for matching, event browsing and roster generation.

Proposed benchmarks, not validated: browse < 2 s; matching for ≤ 500 candidates < 5 s; roster draft < 10 s. [P]

Monitoring [P]: API request logs and error tracking; match/roster run durations; notification delivery

status; emergency alert-to-confirmation timings; Agent activity logs (latency, model, failure rate) in Firebase `agentActivity`; LLM/Maps/Email usage and cost tracking; health checks for the Agent service. Tooling choice: TBD.

B13. Deployment

Component	Deployment
React/Vite frontend	Vercel
Node/Express API	Vercel (serverless functions) or an equivalent Node host if needed [P]
Python Agent Service	Deployed on an environment suitable for its workload — a long-running/containerized service host — rather than forced into an unsuitable serverless execution model. Provider: TBD (A17 #11); responsibility is separated from the Express API.
Supabase / Firebase / Clerk / Maps / Email / n8n	Managed services

The Express API and the Agent Service communicate over an authenticated HTTP interface, so the Agent service can be moved or scaled without touching the core API. Environment configuration (keys, model config, weights) is managed per environment.

B14. Technical Risks

Risk	Mitigation
Serverless time limits for LLM calls	Agent service on a suitable host; async/streaming responses
Two-datastore consistency	Supabase-first writes; Firebase derived/rebuildable; ID references only
Real-time layer failure during emergency	API polling fallback; Supabase remains authoritative
LLM hallucination	Grounded context, basis citations, insufficient-data fallback, no write tools
Matching disputes ("why wasn't I picked?")	Stored match runs/results and weights; explanations
Geo query performance	PostGIS indexing; pre-filter by bounding box
Third-party API cost/limits	Rate limits, caching, usage monitoring
Notification delivery failures	Delivery status tracking; in-app inbox as record; retry
Time constraints	Tiered MVP and cut order (B15)

B15. Development Roadmap

Phase	Focus	Deliverable
0	Setup: repo, Vercel, Clerk, Supabase, Firebase, seed-data plan; resolve A17 decisions	Auth working, skeleton deployed
1	Profiles: schema, volunteer and NGO profiles	Profile CRUD, roles
2	Events: CRUD, requirements, shifts, public discovery, registration, Maps	Public discovery with distance filter
3	Matching and roster: gates, scoring, explanations, audit tables, roster draft/conflicts	Match & roster UI
4	Tracking: attendance, Karma ledger, reliability records, achievements	Verified history
5	Transparency and Agent: NGO indicators page; Python Agent service with Firebase state	Profile page, Agent chat
6	Dashboard and demo polish; seeded data; rehearsals	Primary demo flow working
7 (optional)	Emergency alert → confirm → deploy; notifications via n8n/email; advanced analytics	Secondary demo

Cut order if time runs short: advanced analytics → burnout indicators → emergency enhancements → advanced notifications → backups → Hugging Face usage → UI polish. The primary demo flow (A18) is protected.

